

AI Receptionist Console

Product Requirements Document · v2.0 — built

Author: Shamitha

Live: super-front-desk.sosham.me

Contents

1. Where this comes from — the field research
 2. The problem
 3. What I built
 4. How each part of the problem is solved
 5. The five screens
 6. The rules the product enforces
 7. How anyone knows it works
 8. What I'd do next
 9. What my evidence covers — and doesn't
-

1. Where this comes from

This design comes from working in healthcare and watching and working with call centre agents and see what works and fails in specific, repeatable ways.

1.1 What I did

I led Phase 2 of patient relationship management discovery at Narayana Health — a hospital group running one of the larger healthcare call centre operations in India.

Method	Ethnographic observation during live calls, semi-structured stakeholder interviews, workflow mapping with team leads
Depth	I sat beside agents while they took real patient calls.
Breadth	Five distinct agent types, each with a different workflow and a different tool

Longitudinal	Phase 2 revalidated ~40 gaps found earlier — so I can say which problems were fixed, which persisted, and which evolved into new ones
Systems depth	I mapped the full integration landscape underneath: CTI telephony, the hospital management system, CRM, marketing automation, social/ORM ingestion, and the middleware between them

Scale of the operation studied: five agent types across inbound, clinical escalation, outbound leads, unconverted-call recovery, and hospital-side closure. The reputation team alone handled 400–500 tickets a day.

1.2 Why this matters for a dental phone product

The failures I observed are **properties of call-handling systems**, not properties of Indian healthcare. The same six failures appear whenever a conversation has to survive a handoff — which is exactly what an AI receptionist has to do.

I have been explicit in §9 about where this transfers and where it doesn't.

1.3 The six findings this product is built on

	Finding	What I saw
F1	Context dies at every handoff	The highest-value flow was surgical enquiries. A first-line agent transferred to a clinical specialist, who booked and dropped off. The hospital unit team then had to convert that booking into an admission — with no record of what the specialist discussed . They started blind and re-asked the patient everything. At the most valuable point in the entire funnel.
F2	Every agent kept a private notes file	All five teams. Every call. Not habit — the system lost unsaved context on tab switch, so agents built their own memory outside it. Nobody was measuring the workaround.
F3	80% of calls were logged as free typed text	Only one flow captured a structured category. Every conversion number the business reported was therefore unreliable — and

	Finding	What I saw
		nobody found out until someone tried to build a report on it.
F4	Agents without domain literacy transfer everything	First-line agents couldn't classify a surgical query, so they escalated every clinical call — adding delay to the calls that mattered most and consuming the scarcest people.
F5	Partial integration doesn't partially solve the problem	Booking was integrated for outpatient consultations only. Health checkups, scans and home collection still forced agents into a second application mid-call. Connecting one service type out of four left agents in the old system most of the day.
F6	Scarcity plus language mismatch equals abandoned calls	Around ten clinical specialists. When none was free, or the caller's language didn't match, the call sat open and the patient waited.

Every one of these is the same sentence: context does not survive a handoff — not between tabs, not between teams, not between systems.

That sentence is the product.

2. The problem

A dental practice loses money through its phone and cannot see it happening.

The front desk is one or two people answering calls while a patient stands in front of them. Calls are missed after hours, at lunch, and whenever two things happen at once. A phone line doesn't report what it didn't answer, so **nobody in the building knows the number.**

And the value is uneven. A missed cleaning is worth a few hundred dollars. A missed implant case is worth thousands. The practice can't tell them apart, because it never heard either one.

The sentence no practice can say today:

"Forty-one people called about implants last quarter. Eleven booked. Here are the other thirty."

Making that sentence sayable is the product.

The hard part isn't answering

Answering is the easy half. The hard half:

The agent has to book into **a live schedule it doesn't own**, inside **software it may not be allowed to write to**, and hand off to a human **without the caller starting over**.

Get that wrong and two patients arrive for one slot. Then the practice manager switches it off, and no amount of conversational quality brings her back.

3. What I built

A working console at **super-front-desk.sosham.me** — five screens where a practice watches the agent, controls it, and checks it.

Screen	What it's for	Who lives there
Today	What the phone did, what needs a human	Front desk, owner
Calls	Every call, searchable, with the evidence trail	Front desk
Approvals	Slots the agent held, waiting for a human	Practice manager
Agent	What the agent is permitted to do	Practice manager
Quality	Run accuracy checks and track them over time	Practice manager, ops

4. How each part of the problem is solved

This is the section to read if you read only one.

4.1 The problem statement, answered

The ask	How it's solved	Where to see it	Grounded in
Answer the calls nobody is answering	Agent picks up after hours and on overflow; every call produces a record whether or not it books	Today, Calls	F6
Turn each call into data the practice can act on	Fixed fields with fixed options — never a free paragraph. Opportunity, intent, treatment type, readiness, reason not booked, insurance, flags	Call drawer	F3
Book into a schedule the agent doesn't own	The practice defines a constraint set; the agent can only write inside it. Soft hold, conditional write, idempotency key	Agent screen	F5
Work inside software it may not be able to write to	Capability registry with two axes — what the system supports, what the practice enabled. The agent changes what it <i>says out loud</i> accordingly	Agent screen	F5
Hand off without the caller starting over	Context card delivered to the human <i>before</i> they pick up. Handoff is one-directional by design	Call drawer	F1
Don't create a mess a human has to clean up	Two hard gates: zero double-bookings, zero silent overwrites. Blackout types can never be booked in any mode	Agent screen	F2
Prove it works	Quality screen: run checks, see failures, track scores across runs, compare two runs	Quality	F3

4.2 Six findings → six design decisions

Finding	Design decision it produced
F1 — context dies at handoff	One-directional escalation with a context card. The human sees who's calling, what they want, and <i>what's already been promised</i> , before picking up. Human-to-bot handback is excluded on purpose — the agent has no record of what the human said, so handing back recreates the exact failure I watched.
F2 — private notes as survival	Conversation state is first-class and saves without being asked. A voice agent has no notepad to fall back on. And override rate is treated as a metric — at the hospital, nobody was measuring the workaround, so nobody knew trust had broken.
F3 — free text destroys reporting	Closed lists, never prose. Every field has a fixed set of allowed values. This is also why the Quality screen exists — a number nobody can check is the same failure in a new place.
F4 — no literacy means escalate everything	"Is this a possible patient?" is the load-bearing classification. A supply vendor mentioning "implant" is not an opportunity. Get this wrong and every conversion number is meaningless.
F5 — partial integration fails	Capability is not a yes/no. Two axes, tracked separately, because <i>"your system can't"</i> and <i>"you turned it off"</i> are different conversations.
F6 — scarcity strands callers	An honest commitment beats an optimistic hold. <i>"Someone will call you at 9am"</i> instead of silence. Held slots have a visible expiry and notify someone when they lapse.

5. The five screens

5.1 Today

Four numbers, then a worklist.

The numbers: calls answered, how many the agent handled alone, opportunities found, and **value at risk** — the money sitting in calls nobody has returned. That last one is what the owner came for.

Below: **Needs you** — a worklist sorted by urgency and value, never chronologically. A list sorted by time is a feed, and feeds don't get finished.

When there's nothing to do, the screen says so plainly: *"Nothing needs you. The agent handled all 47 calls."*

5.2 Calls

Every call, filterable. Clicking opens a side panel rather than a new page, so reviewing twenty calls doesn't lose your place.

Three things in the panel worth noting:

Confidence shown as dots, not decimals. A practice manager can't act on "0.87" but can act on "four out of five."

Flags link to the moment. Click a flag and the transcript scrolls to the exact turn where it was heard. This turns *"the AI said so"* into *"here's where it heard it."*

"Add to quality set." One click turns this real call into a permanent test case. Found the agent getting something wrong? It's now a test that runs forever. This closes the loop between the Calls screen and the Quality screen, and nothing in the dental category does it.

5.3 Approvals

Slots the agent held, waiting for a human.

Expiry is the most prominent thing on the card after the slot itself, turning red under fifteen minutes. **A hold that expires silently is a broken promise to a patient** — that's F6 again, in a new place.

At the bottom, one quiet line:

Approved without changes: 47 of 52 (90%). Practices usually switch to direct booking above 85%.

That's the commercial engine of the product, stated as a fact rather than an upsell.

5.4 Agent

Where the practice decides what the agent may do. Three booking modes, side by side — **and each card shows what the caller actually hears**:

Takes details only	Asks you first	Books directly
<i>"The front desk will call you back."</i>	<i>"I've held Tuesday at 2:40 — the office will confirm."</i>	<i>"You're booked for Tuesday at 2:40."</i>

The first two are not promises. The third is. **Showing the sentence makes that difference impossible to miss** — and saying the third when you mean the second is how a patient ends up at a locked door.

Below: the capability table with its two columns, and the never-book list which overrides every other setting.

5.5 Quality — the one nobody else has

One button: **Run the check**. Nothing else on the screen, because the person who most needs to run it is not an engineer.

It runs a set of calls where the correct answer is already known, and reports:

- A plain verdict — *"22 of 24 correct"*
- Per-check scores against targets, in plain questions: *"Did it work out what kind of treatment?"*
- **The failures, expanded by default** — what it should have said, what it said instead, and why. Never make someone hunt for the failures.

It tracks — this is the part that matters:

- Every run is saved, dated, and can be named (*"after we changed the greeting"*)
- Each score shows the change since last time
- Two runs can be compared, showing **exactly which calls got better and which got worse**
- Regressions are listed first

Why tracking is the whole point: without history, you change something, the number moves, and you have no idea whether that was you.

One check is treated differently. Emergency detection isn't a target to hit — it's marked *must always pass*, in its own full-width row. A false alarm costs thirty seconds of a human's time. A miss is a patient with facial swelling told to call back Monday.

6. The rules the product enforces

6.1 The AI never decides to touch the schedule

The system is split in two:

The AI does	Plain code does
Understand the call	Decide whether to book
Fill in the fields	Check the rules
Say how confident it is	Write to the system

An AI mistake becomes a wrong note, fixed in ten seconds. It can never become a wrong booking, which costs a patient. The failure lands on the cheap side.

This is the single most important design decision in the product, and it's visible in the prototype — the decision layer is ordinary deterministic code, not a prompt instruction.

6.2 Never promise what the system can't deliver

The agent looks up what it can do at this practice and changes what it says. The difference between *"you're booked"* and *"the office will confirm"* is the difference between a renewal and a refund.

6.3 Two rules that never bend

1. **Zero double-bookings**
2. **Zero silent overwrites of something a human typed**

Not "very few." Zero.

Why only two? If every rule is non-negotiable, then at 6pm on a Friday with a release waiting, all of them quietly become negotiable. **Naming the two that never move is what makes them hold.**

7. How anyone knows it works

Measure	Target
Calls answered that would have been missed	>98%
Calls handled without a human	55%
Bookings needing a human to fix	<0.5%
Double-bookings	0
Approvals accepted without changes	>85% before offering direct booking
Escalated calls where the human re-asked basics	<10%

Containment is reported but never optimised directly. The cheapest way to raise it is to stop escalating calls that should be escalated.

The trust ladder

Adoption is a sequence, and the product is built to walk it:

Stage	Mode	What earns the next step
1. Watch	Takes details only	The practice sees calls it was missing
2. Check	Asks first	Approvals accepted unchanged above 85%
3. Trust	Books directly	Thirty days, zero schedule problems
4. Extend	Also during business hours	Front desk says escalations arrive usable

Stage 2 is where it's won or lost. That's where the number that convinces the practice manager gets generated — and the product generates it automatically rather than asking anyone to trust a sales deck.

8. What I'd do next

1. **Call the competitors' demo lines.** Book, then call back to reschedule, then call back to cancel. This is the only honest way to assess the category and I'd do it in week one.
2. **Replace synthetic test calls with consented production calls**, de-identified, stratified so no category falls below twenty examples.
3. **Two labellers per call plus a third to adjudicate**, and report how often the humans agree. If two trained people can't agree what a call was about, that's a category definition problem, not a model problem — and no amount of prompting fixes it.
4. **Build the per-location capability report** for multi-site groups, so implementation teams know before go-live what each site can actually do.
5. **Instrument override rate** as a first-class metric. It's the signal that moves before any dashboard does, and it costs nothing to collect.

Open questions I'd want answered

- Who tells the patient when the agent books and the practice later cancels?
 - Should the capability report be shown to customers before they buy? I think yes — it's the honest thing, and it's the implementation team's best tool — but it exposes gaps to competitors.
 - Who staffs the approval queue at a two-person practice on the day one of them is out?
 - Should the caller be told they're speaking to an AI, and what does that disclosure cost? Worth measuring rather than assuming.
-

9. What my evidence covers — and doesn't

Stating this plainly, because the boundary matters more than the volume.

Transfers well. Handoff information loss, live-call memory, structured disposition, classification-driven routing, integration scope, and scarcity-driven abandonment. These are properties of call-handling systems, and **I observed them identically across five different agent types.**

Transfers partially. Scale. I studied a centralised call centre with dedicated agents and team leads. A dental front desk is one or two people also checking patients in. Same failure shapes, different coping mechanisms — and a two-person practice has far less slack to absorb a mistake.

Does not transfer. US dental economics — hygiene recall as the profit engine, case acceptance, and what a new patient is worth under different plan types. Group practice buying

behaviour. Hands-on experience of the specific practice management systems, which I've read about but not built against.

Nothing in this document depends on the parts that don't transfer. Where dental-specific economics would change a decision — pricing, treatment weighting, what counts as high value — I've left it as an open question rather than guessed.

Appendix — research to product, at a glance

What I observed	What it became
Specialist booked, dropped off, unit team started blind	Context card before pickup; one-directional handoff
Every agent running a private notes file	State that saves without being asked; override rate as a metric
80% of calls logged as free text; reporting built on sand	Closed lists everywhere; the Quality screen
Agents escalating every clinical call for lack of literacy	"Is this a possible patient?" as the load-bearing classification
Booking integrated for one service type out of four	Capability registry with two axes
Calls sitting open while patients waited	Honest commitment over optimistic hold; visible hold expiry
Workarounds becoming permanent once trust broke	Two hard gates; the trust ladder